Объектно-ориентированное программирование

4 семестр
Лекция 5.

Содержание лекции:

- Параллельное программирование (TPL)
- Класс Task
- Класс Parallel
- Отмена задач

Преподаватель курса: Клюкин Даниил Анатольевич, ст. преподаватель каф. ПМиИТ

Класс Task

В библиотеке классов .NET задача представлена специальным классом - классом **Task**, который находится в пространстве имен **System.Threading.Tasks**. Данный класс описывает отдельную задачу, которая запускается асинхронно в одном из потоков из пула потоков. Хотя ее также можно запускать синхронно в текущем потоке.

Первый способ создание объекта **Task** и вызов у него метода **Start**:

```
Task task = new Task(
                () => Console.WriteLine("Hello Task!"));
task.Start();
```

Второй способ заключается в использовании статического метода **Task.Factory.StartNew()**. Этот метод также в качестве параметра принимает делегат **Action**, который указывает, какое действие будет выполняться. При этом этот метод сразу же запускает задачу:

```
Task task = Task.Factory.StartNew(
                () => Console.WriteLine("Hello Task!"));
```

Третий способ определения и запуска задач представляет использование статического метода **Task.Run()**:

```
Task task = Task.Run(() => Console.WriteLine("Hello Task!"));
```

Метод **Task.Run()** также в качестве параметра может принимать делегат Action - выполняемое действие и возвращает объект Task.

Ожидание завершения задачи

Чтобы приложение ожидало завершения задачи, можно использовать метод **Wait()** объекта Task:

```
Task task1 = new Task(() => Console.WriteLine("Task1 is
executed"));
task1.Start();

Task task2 = Task.Factory.StartNew(() => Console.WriteLine("Task2
is executed"));

Task task3 = Task.Run(() => Console.WriteLine("Task3 is
executed"));

task1.Wait();   // ожидаем завершения задачи task1
task2.Wait();   // ожидаем завершения задачи task2
task3.Wait();   // ожидаем завершения задачи task3
```

Стоит отметить, что метод Wait() блокирует вызывающий поток, в котором запущена задача, пока эта задача не завершит свое выполнение.

Например:

```
Console.WriteLine("Main Starts");
// создаем задачу
Task task1 = new Task(() =>
{
    Console.WriteLine("Task Starts");
    Thread.Sleep(1000);     // задержка на 1 секунду – имитация
долгой работы
    Console.WriteLine("Task Ends");
});
task1.Start();  // запускаем задачу
task1.Wait();   // ожидаем выполнения задачи
Console.WriteLine("Main Ends");
```

```
Main Starts
Task Starts
Task Ends
Main Ends
```

Для эмуляции долговременной работы здесь в задаче task1 устанавливается задержка на 1 секунду. В итоге, когда выполнение дойдет до вызова task1.Wait() основной поток остановит свое выполнение и будет ждать завершения задачи.

Свойства класса Task

Status: возвращает статус задачи. Представляет перечисление System.Threading.Tasks.TaskStatus, которое имеет следующие значения:

- **Canceled**: задача отменена;
- **Created**: задача создана, но еще не запущена;
- **Faulted**: в процессе работы задачи произошло исключение;
- **RanToCompletion**: задача успешно завершена;
- **Running**: задача запущена, но еще не завершена;
- **WaitingForActivation**: задача ожидает активации и постановки в график выполнения;
- **WaitingForChildrenToComplete**: задача завершена и теперь ожидает завершения прикрепленных к ней дочерних задач;
- **WaitingToRun**: задача поставлена в график выполнения, но еще не начала свое выполнение;

IsCompleted: возвращает true, если задача завершена;

IsCanceled: возвращает true, если задача была отменена;

IsFaulted: возвращает true, если задача завершилась при возникновении исключения;

IsCompletedSuccessfully: возвращает true, если задача завершилась успешно.

Пример:

```
Task task1 = new Task(() =>
{
    Console.WriteLine(
     $"Task{Task.CurrentId} Starts");
    Thread.Sleep(1000);
    Console.WriteLine(
     $"Task{Task.CurrentId} Ends");
});

task1.Start(); //запускаем задачу

// получаем информацию о задаче
Console.WriteLine($"task1 Id: {task1.Id}");
Console.WriteLine($"task1 is Completed: {task1.IsCompleted}");
Console.WriteLine($"task1 Status: {task1.Status}");

task1.Wait(); // ожидаем завершения задачи
```

```
task1 Id: 8
Task8 Starts
task1 is Completed: False
task1 Status: Running
Task8 Ends
```

Вложенные задачи

Одна задача может запускать другую - вложенную задачу. При этом эти задачи выполняются независимо друг от друга. Например:

```
var outer = Task.Factory.StartNew(() =>      // внешняя задача
{
    Console.WriteLine("Outer task starting...");
    var inner = Task.Factory.StartNew(() =>  // вложенная задача
    {
        Console.WriteLine("Inner task starting...");
        Thread.Sleep(2000);
        Console.WriteLine("Inner task finished.");
    });
});
// ожидаем выполнения внешней задачи
outer.Wait();
Console.WriteLine("End of Main");
```

```
Outer task starting...
End of Main
Inner task starting...
```

Вложенная задача может завершиться даже после завершения метода Main.

Если необходимо, чтобы вложенная задача выполнялась как часть внешней, необходимо использовать значение TaskCreationOptions.AttachedToParent:

```
var outer = Task.Factory.StartNew(() =>      // внешняя задача
{
    Console.WriteLine("Outer task starting...");
    var inner = Task.Factory.StartNew(() =>  // вложенная задача
    {
        Console.WriteLine("Inner task starting...");
        Thread.Sleep(2000);
        Console.WriteLine("Inner task finished.");
    }, TaskCreationOptions.AttachedToParent);
});

outer.Wait(); // ожидаем выполнения
Console.WriteLine("End of Main");
```

```
Outer task starting...
Inner task starting...
Inner task finished.
End of Main
```

Внешняя задача завершится когда завершатся все прикрепленные к ней вложенные задачи.

Массив задач

Также как и с потоками, мы можем создать и запустить массив задач. Можно определить все задачи в массиве непосредственно через объект **Task**:

```
Task[] tasks1 = new Task[3]
{
    new Task(() => Console.WriteLine("First Task")),
    new Task(() => Console.WriteLine("Second Task")),
    new Task(() => Console.WriteLine("Third Task"))
};
// запуск задач в массиве
foreach (var t in tasks1)
    t.Start();
```

ИЛИ

```
Task[] tasks2 = new Task[3];
int j = 1;
for (int i = 0; i < tasks2.Length; i++)
    tasks2[i] = Task.Factory.StartNew(() =>
Console.WriteLine($"Task {j++}"));
```

Вопрос: Что отобразится на экране?

Исправим это с помощью Task.WaitAll():

```
Task[] tasks = new Task[3];
for (var i = 0; i < tasks.Length; i++)
{
    tasks[i] = new Task(() =>
    {
        Thread.Sleep(1000); // эмуляция долгой работы
        Console.WriteLine($"Task{i} finished");
    });
    tasks[i].Start();   // запускаем задачу
}
Console.WriteLine("Завершение метода Main");

Task.WaitAll(tasks); // ожидаем завершения всех задач
```

```
Завершение метода Main
Task3 finished
Task3 finished
Task3 finished
```

В примере выше мы столкнулись с такой вещью как **замыкание** (нам 3 раза вывело Task**3**, хотя i менялось от 0 до 2).

Замыкание это закрепление объекта за его окружением (областью видимости).

Исправим пример выше создав локальную переменную index:

```
Task[] tasks = new Task[3];
for (var i = 0; i < tasks.Length; i++)
{
    var index = i;
    tasks[i] = new Task(() =>
    {
        Thread.Sleep(1000); // эмуляция долгой работы
        Console.WriteLine($"Task{index} finished");
    });
    tasks[i].Start();   // запускаем задачу
}
Console.WriteLine("Завершение метода Main");

Task.WaitAll(tasks); // ожидаем завершения всех задач
```

```
Завершение метода Main
Task2 finished
Task0 finished
Task1 finished
```

Получение результатов из задач

```
int n1 = 4, n2 = 5;
Task<int> sumTask = new Task<int>(() => Sum(n1, n2));
sumTask.Start();

int result = sumTask.Result;
Console.WriteLine($"{n1} + {n2} = {result}"); // 4 + 5 = 9

int Sum(int a, int b) => a + b;
```

```
4 + 5 = 9
```

При обращении к свойству **Result** текущий поток останавливает выполнение и ждет, когда будет получен результат из выполняемой задачи.

Задачи продолжения

Задачи продолжения или **continuation task** позволяют определить задачи, которые выполняются после завершения других задач.

Благодаря этому мы можем вызвать после выполнения одной задачи несколько других, определить условия их вызова, передать из предыдущей задачи в следующую некоторые данные.

Задачи продолжения похожи на методы обратного вызова, но фактически являются обычными задачами **Task**.

Посмотрим на примере:

```
Task task1 = new Task(() =>
{
    Console.WriteLine($"Id задачи: {Task.CurrentId}");
});

// задача продолжения - task2 выполняется после task1
Task task2 = task1.ContinueWith(PrintTask);

task1.Start();
// ждем окончания второй задачи
task2.Wait();
Console.WriteLine("Конец метода Main");
void PrintTask(Task t)
{
    Console.WriteLine($"Id задачи: {Task.CurrentId}");
    Console.WriteLine($"Id предыдущей задачи: {t.Id}");
    Thread.Sleep(3000);
}
```

```
Id задачи: 9
Id задачи: 8
Id предыдущей задачи: 9
Конец метода Main
```

Первая задача задается с помощью лямбда-выражения, которое просто выводит id этой задачи. Вторая задача - задача продолжения задается с помощью метода **ContinueWith**, который в качестве параметра принимает делегат Action<Task>. То есть метод **PrintTask**, который передается в вызов ContinueWith, должен принимать параметр типа Task.

Можно было написать так:

```
Task task2 = task1.ContinueWith(t => PrintTask(t));
```

Также мы можем передавать конкретный результат работы предыдущей задачи:

```
Task<int> sumTask = new Task<int>(() => Sum(4, 5));
// задача продолжения
Task printTask = sumTask.ContinueWith(task =>
   PrintResult(task.Result));

sumTask.Start();
// ждем окончания второй задачи
printTask.Wait();
Console.WriteLine("Конец метода Main");

int Sum(int a, int b) => a + b;
void PrintResult(int sum) => Console.WriteLine($"Sum: {sum}");
```

```
Sum: 9
Конец метода Main
```

Рассмотрим пример параллельного вычисления интеграла.

$$\int_a^b f(x)dx = \sum_{i=1}^{N} \int_{a_i}^{b_i} f(x)dx$$

```
double Integrate(double from, double to, int intervals = 100)
{
    var integral = 0.0;
    var step = (to - from) / intervals;
    for (int i = 0; i < intervals; i++) {
        var xLeft = from + i * step;
        var xRight = xLeft + step;
        integral += (f(xLeft) + f(xRight)) / 2 * step;
    }

    return integral;
}
double f(double x) => x * x;
```

```
var from = 0.0;
var to = 1.0;

//Разобьём интеграл на множество интегралов, равное количеству
ядер ЦП
var integralTasks = new Task<double>[Environment.ProcessorCount];

var step = (to - from) / integralTasks.Length;

for (var i = 0; i < integralTasks.Length; i++)
{
    var xLeft = from + i * step;
    var xRight = xLeft + step;
    integralTasks[i] = Task.Run(() => Integrate(xLeft, xRight,
100000000));
}
Task.WaitAll(integralTasks);
Console.WriteLine(integralTasks.Sum(t => t.Result));
```

```
0,3333333333333414
```

Класс Parallel

Класс **Parallel** также является частью TPL и предназначен для упрощения параллельного выполнения кода. **Parallel** имеет ряд методов, которые позволяют распараллелить задачу.

```
Parallel.Invoke(// метод Parallel.Invoke выполняет три метода
    Print,
    () =>
    {
        Console.WriteLine($"Выполняется задача {Task.CurrentId}");
        Thread.Sleep(3000);
    },
    () => Square(5)
);
```

```
Выполняется задача 13
Выполняется задача 11
Выполняется задача 12
Результат 25
```

```
void Print() {
    Console.WriteLine($"Выполняется задача {Task.CurrentId}");
    Thread.Sleep(3000);
}
void Square(int n) {// вычисляем квадрат числа
    Console.WriteLine($"Выполняется задача {Task.CurrentId}");
    Thread.Sleep(3000);
    Console.WriteLine($"Результат {n * n}");
}
```

```
Выполняется задача 13
Выполняется задача 11
Выполняется задача 12
Результат 25
```

```
Parallel.For(int, int, Action<int>)
```

Метод **Parallel.For** позволяет выполнять итерации цикла параллельно. Первый параметр метода задает начальный индекс элемента в цикле, а второй параметр - конечный индекс. Третий параметр - делегат Action - указывает на метод, который будет выполняться один раз за итерацию:

```
Parallel.For(1, 5, Square);

// вычисляем квадрат числа
void Square(int n)
{
    Console.WriteLine($"Выполняется задача {Task.CurrentId}");
    Console.WriteLine($"Квадрат числа {n} равен {n * n}");
    Thread.Sleep(2000);
}
```

```
Выполняется задача 9
Выполняется задача 8
Квадрат числа 3 равен 9
Выполняется задача 11
Квадрат числа 4 равен 16
Выполняется задача 10
Квадрат числа 1 равен 1
Квадрат числа 2 равен 4
```

```
Parallel.ForEach<TSource>(IEnumerable<TSource>
source,Action<TSource> body)
```

Метод Parallel.ForEach осуществляет итерацию по коллекции, реализующей интерфейс IEnumerable, подобно циклу foreach, только осуществляет параллельное выполнение перебора.

```
ParallelLoopResult result = Parallel.ForEach<int>(
       new List<int>() { 1, 3, 5, 8 },
       Square
);

// вычисляем квадрат числа
void Square(int n)
{
    Console.WriteLine($"Выполняется задача {Task.CurrentId}");
    Console.WriteLine($"Квадрат числа {n} равен {n * n}");
    Thread.Sleep(2000);
}
```

```
Выполняется задача 11
Выполняется задача 10
Квадрат числа 5 равен 25
Выполняется задача 9
Квадрат числа 8 равен 64
Выполняется задача 8
Квадрат числа 1 равен 1
Квадрат числа 3 равен 9
```

Выход из цикла

В стандартных циклах for и foreach предусмотрен преждевременный выход из цикла с помощью оператора **break**. В методах Parallel.ForEach и Parallel.For мы также можем, не дожидаясь окончания цикла, выйти из него:

```
ParallelLoopResult result = Parallel.For(1, 25, Square);

if (!result.IsCompleted)
    Console.WriteLine($"Выполнение цикла завершено на итерации
        {result.LowestBreakIteration}");

// вычисляем квадрат числа
void Square(int n, ParallelLoopState pls)
{
    // если передано 5, выходим из цикла
    if (n == 5) pls.Break();
    Console.WriteLine($"Квадрат числа {n} равен {n * n}");
    Thread.Sleep(2000);
}
```

Вывод на консоль:

```
Квадрат числа 7 равен 49
Квадрат числа 3 равен 9
Квадрат числа 1 равен 1
Квадрат числа 5 равен 25
Квадрат числа 9 равен 81
Квадрат числа 11 равен 121
Квадрат числа 13 равен 169
Квадрат числа 15 равен 225
Квадрат числа 17 равен 289
Квадрат числа 19 равен 361
Квадрат числа 21 равен 441
Квадрат числа 2 равен 4
Квадрат числа 4 равен 16
Выполнение цикла завершено на итерации 5
```

Отмена задач и параллельных операций. CancellationToken

Параллельное выполнение задач может занимать много времени. И иногда может возникнуть необходимость прервать выполняемую задачу. Для этого платформа .NET предоставляет структуру **CancellationToken** из пространства имен System.Threading.

Общий алгоритм отмены задачи обычно предусматривает следующий порядок действий:

1. Создание объекта **CancellationTokenSource**, который управляет и посылает уведомление об отмене токену.
2. С помощью свойства **CancellationTokenSource.Token** получаем токен и передаем его в задачу, которая может быть отменена.
3. Определяем в задаче действия на случай ее отмены.
4. Вызываем метод **CancellationTokenSource.Cancel**(), который устанавливает для свойства **CancellationToken.IsCancellationRequested** значение true.
5. когда работа с объектом **CancellationTokenSource** завершена, у него следует вызвать метод **Dispose** для освобождения всех связанных с ним используемых ресурсов.

```
CancellationTokenSource cancelTokenSource = new
                                        CancellationTokenSource();
CancellationToken token = cancelTokenSource.Token;
Task task = new Task(() => // задача вычисляет квадраты чисел
{
    for (int i = 1; i <= 10; i++)
    {
        if (token.IsCancellationRequested)  // проверяем наличие
сигнала отмены
        {
            Console.WriteLine("Операция прервана");
            return;     //  выходим из метода и тем самым
завершаем задачу
        }
        Console.WriteLine($"Квадрат числа {i} равен {i * i}");
        Thread.Sleep(200);
    }
}, token);
```

```
task.Start();
Thread.Sleep(1000); // после задержки по времени отменяем
выполнение задачи
cancelTokenSource.Cancel();
task.Wait(); // ожидаем завершения задачи
Console.WriteLine($"Task Status: {task.Status}"); //  проверяем
статус задачи
cancelTokenSource.Dispose(); // освобождаем ресурсы
```

Вывод на консоль:

```
Квадрат числа 1 равен 1
Квадрат числа 2 равен 4
Квадрат числа 3 равен 9
Квадрат числа 4 равен 16
Квадрат числа 5 равен 25
Операция прервана
Task Status: RanToCompletion
```